Approach and core components

6 min read Updated 4 Jun 2026 Topic 72 of 77

Bookmark Mark done

24A. Machine Coding Strategy + 24B. Core UI Components

Q 24.1 How to approach a machine coding round — time management, structure, scoring

Time breakdown for a 90-minute round:

JAVASCRIPT Copy

```
0–5 min:   Read requirements. Clarify scope. Ask: "Is accessibility expect
           Should I handle edge cases like empty states?" Plan your comp
5–15 min:  Set up folder structure. Create the component skeleton. Get t
           render working (even if it's just placeholder HTML).
15–55 min: Build core functionality. Get the WORKING version first — no
           no edge cases, just the feature working end-to-end.
55–75 min: Add edge cases, error handling, keyboard navigation, loading
75–85 min: Polish — CSS cleanup, responsive tweaks, code cleanup.
85–90 min: Test by using it. Walk through your code mentally.
```

What evaluators score:

1. **Working code (40%)** — Does it actually work? Can the evaluator click through the feature? A polished but broken component scores lower than an ugly but functional one.

3. **Edge cases (15%)** — Empty states, error states, very long text, rapid clicking, boundary values.
4. **Accessibility (10%)** — Keyboard navigation, ARIA attributes, focus management.
5. **CSS/Design (10%)** — Reasonable layout, responsive behavior, doesn't look broken.
6. **Bonus (5%)** — Tests, TypeScript, performance considerations.

Common mistakes that cost candidates the round:

- Spending 30 minutes on perfect CSS before the feature works
- Building everything from scratch instead of using semantic HTML (`<dialog>`, `<details>`)
- Not handling the empty state (list with zero items)
- Forgetting keyboard navigation entirely
- Over-engineering state management for a simple component

Q 24.2 Accordion / Collapsible Panel

Requirements: Multiple collapsible sections. Click header to toggle content. Optional: only one section open at a time (exclusive mode).

JSX Copy

```jsx
function Accordion({ items, exclusive = false }) {
  const [openIds, setOpenIds] = useState(new Set());

  function toggle(id) {
    setOpenIds(prev => {
      const next = new Set(exclusive ? [] : prev);
      if (prev.has(id)) next.delete(id);
      else next.add(id);
      return next;
```

```
  return (
    <div className="accordion">
      {items.map(item => (
        <div key={item.id} className="accordion-item">
          <button
            onClick={() => toggle(item.id)}
            aria-expanded={openIds.has(item.id)}
            aria-controls={`panel-${item.id}`}
          >
            {item.title}
            <span className={`chevron ${openIds.has(item.id) ? 'open' : ''
          </button>
          <div id={`panel-${item.id}`} role="region" hidden={!openIds.has(
            {item.content}
          </div>
        </div>
      ))}
    </div>
  );
}
```

Edge cases: Empty items array, very long content, animated open/close (`grid-template-rows: 0fr → 1fr`).

Accessibility: `aria-expanded` on trigger, `aria-controls` linking button to panel, `role="region"` on panel. Native `<details>` / `<summary>` does this for free — mention it.

Q 24.3 Modal / Dialog with Focus Trap

Requirements: Overlay with backdrop, close on Escape, close on backdrop click, focus trapped inside, return focus on close.

JSX Copy

```jsx
  useEffect(() => {
    const dialog = dialogRef.current;
    if (isOpen) dialog?.showModal();
    else dialog?.close();
  }, [isOpen]);

  return (
    <dialog ref={dialogRef} onClose={onClose} className="modal">
      <div className="modal-header">
        <h2>{title}</h2>
        <button onClick={onClose} aria-label="Close dialog">×</button>
      </div>
      <div className="modal-body">{children}</div>
    </dialog>
  );
}
```

CSS Copy

```css
dialog::backdrop { background: rgba(0, 0, 0, 0.5); }
dialog { border: none; border-radius: 12px; padding: 24px; max-width: 500p
```

Use `<dialog>` with `.showModal()` — it handles focus trapping, Escape key, backdrop, and top-layer placement for free.

Q 24.4 Tabs Component with Keyboard Navigation

JSX Copy

```jsx
function Tabs({ tabs }) {
  const [activeIndex, setActiveIndex] = useState(0);

  function handleKeyDown(e) {
    let newIndex = activeIndex;
```

```
    if (e.key === 'End') newIndex = tabs.length - 1;
    if (newIndex !== activeIndex) { e.preventDefault(); setActiveIndex(new
  }

  return (
    <div>
      <div role="tablist" onKeyDown={handleKeyDown}>
        {tabs.map((tab, i) => (
          <button key={tab.id} role="tab" aria-selected={i === activeIndex
            aria-controls={`panel-${tab.id}`} tabIndex={i === activeIndex
            onClick={() => setActiveIndex(i)}
            ref={el => { if (i === activeIndex) el?.focus(); }}>
            {tab.label}
          </button>
        ))}
      </div>
      {tabs.map((tab, i) => (
        <div key={tab.id} role="tabpanel" id={`panel-${tab.id}`} hidden={i
          {tab.content}
        </div>
      ))}
    </div>
  );
}
```

Accessibility: Roving tabindex pattern — only active tab in tab order, arrow keys move between tabs.

Q 24.5 Tooltip with Smart Positioning

Show on hover/focus. Smart positioning — check if fits above, else below, else right. Use `getBoundingClientRect()`.

JAVASCRIPT Copy

```
  if (triggerRect.top - tooltipRect.height - gap > 0)
    return { top: triggerRect.top - tooltipRect.height - gap, left: trigge
  if (triggerRect.bottom + tooltipRect.height + gap < window.innerHeight)
    return { top: triggerRect.bottom + gap, left: triggerRect.left };
  return { top: triggerRect.top, left: triggerRect.right + gap };
}
```

Accessibility: `role="tooltip"`, trigger has `aria-describedby`, show on focus too (not just hover).

Q 24.6 Toast / Notification System

JSX Copy

```jsx
const ToastContext = createContext();

function ToastProvider({ children }) {
  const [toasts, setToasts] = useState([]);

  function addToast(message, type = 'info', duration = 5000) {
    const id = crypto.randomUUID();
    setToasts(prev => [...prev, { id, message, type }]);
    if (duration > 0) setTimeout(() => removeToast(id), duration);
  }

  function removeToast(id) {
    setToasts(prev => prev.filter(t => t.id !== id));
  }

  return (
    <ToastContext.Provider value={{ addToast }}>
      {children}
      <div className="toast-container" aria-live="polite">
        {toasts.map(toast => (
          <div key={toast.id} className={`toast toast-${toast.type}`}>
            <span>{toast.message}</span>
```

```
        </div>
      </ToastContext.Provider>
    );
}
```

Accessibility: `aria-live="polite"` on container. Error toasts use `role="alert"`.

Q 24.7 Star Rating Component

JSX Copy

```jsx
function StarRating({ value = 0, onChange, max = 5, readOnly = false }) {
  const [hover, setHover] = useState(0);

  return (
    <div role="radiogroup" aria-label={`Rating: ${value} out of ${max} sta
      {Array.from({ length: max }, (_, i) => i + 1).map(star => (
        <button key={star} role="radio" aria-checked={star === value}
          aria-label={`${star} star${star > 1 ? 's' : ''}`}
          className={star <= (hover || value) ? 'star filled' : 'star'}
          onClick={() => !readOnly && onChange?.(star)}
          onMouseEnter={() => !readOnly && setHover(star)}
          onMouseLeave={() => setHover(0)} disabled={readOnly}>
          ★
        </button>
      ))}
    </div>
  );
}
```

Q 24.8 – Q 24.13 (Progress Bar, Breadcrumb, Pagination, Tag Input, Toggle Switch, Skeleton Loader)

Progress Bar (24.8): `role="progressbar"`, `aria-valuenow`, `aria-valuemin`, `aria-valuemax`. Two types: determinate (width percentage) and indeterminate (CSS animation loop).

Breadcrumb (24.9): `nav` with `aria-label="Breadcrumb"`, `ol` with `li` items, current page has `aria-current="page"`. Last item is not a link.

Pagination (24.10): Show page numbers with ellipsis for large ranges (1 2 … 8 9 10 … 99 100). `aria-current="page"` on active. Disable prev/next at boundaries. Consider `?page=N` URL params.

Tag Input (24.11): Input creates tags on Enter or comma. Tags dismissible with × or Backspace. Prevent duplicates. `aria-label` on each tag's remove button.

Toggle Switch (24.12): `role="switch"`, `aria-checked`. Keyboard: Space to toggle. Visual transition with CSS `transform`.

Skeleton Loader (24.13): Animated placeholder matching content shape. CSS `linear-gradient` animation. `aria-busy="true"` while loading.

QUICK CHECK — no marks, just for you.

1. You are 55 minutes into a 90-minute machine coding round. The core feature works end-to-end but is unstyled, has no empty state, and no keyboard support. Based on the scoring breakdown in this lesson, what did you get right?

- ○ Working code is the largest single scoring bucket at 40%, so getting the feature functional before polish protects the biggest share of the score
- ○ Accessibility and edge cases together outweigh working code, so the current state scores below half
- ○ CSS and design carry the most weight, so leaving styling for last means most of the score is still unearned

Code quality is scored highest at 40%, so structure matters more than whether the feature runs

Check answer

2. In the Tabs component, every tab button gets tabIndex={i === activeIndex ? 0 : -1} rather than leaving all buttons focusable. What does this accomplish?

- It lets screen readers announce the tab count, which aria-selected alone cannot convey
- It implements roving tabindex: only the active tab sits in the Tab order, and arrow keys move focus between tabs
- It stops the tablist keydown handler from firing on tabs that are not currently active
- It prevents inactive tabs from being clicked with the mouse until they are activated

Check answer

3. The Modal example calls dialogRef.current.showModal() instead of hand-rolling an overlay div with a focus trap. According to the lesson, which set of behaviours does that choice give you without writing extra code?

- Focus trapping, Escape-to-close, the backdrop, and top-layer placement
- Focus trapping and Escape-to-close only; the backdrop and stacking still need CSS z-index management
- Backdrop rendering and top-layer placement only; focus must still be trapped manually with a keydown handler
- Returning focus to the trigger and animating open/close, but not Escape handling

Check answer

Previous

Miscellaneous

Next

Form and input components

LEARN MORE

ARTICLES & DOCS

Front-End System Design Playbook
greatfrontend.com

VIDEOS



How to crack front-end coding (LLD)
YouTube



Toast component — HLD & LLD
YouTube



Front-end system design: LLD & HLD
YouTube

Hand-picked references for this lesson.